



*Department of Computer Science and Engineering (CSE)*

*Semester: (Spring, Year: 2026), B.Sc. in CSE*

### Task Scheduler with Activity Selection

*Course Title: Algorithms*

*Course Code: CSE-246*

*Section: 9*

### Students Details

| Name                   | ID            |
|------------------------|---------------|
| Rayhan Bin Jalal       | 2023-3-60-612 |
| Md. Wasi-Uz-Zaman      | 2024-1-60-068 |
| Md. Jishan Alam Shanto | 2024-1-60-076 |

*Submission Date: 18-4-2026*

*Teacher's Name: Rabea Khatun*

[For teachers use only: **Don't write anything inside this box**]

| <u>Lab Project Status</u> |            |
|---------------------------|------------|
| Marks:                    | Signature: |
| Comments:                 | Date:      |

## Table of Contents

|                                                 |   |
|-------------------------------------------------|---|
| 1. Introduction:                                | 3 |
| 1.1 Overview                                    | 3 |
| 1.2 Motivation                                  | 3 |
| 1.3 Goals:                                      | 3 |
| 2. Design & Implementation:                     | 4 |
| 2.1 Project Details:                            | 4 |
| 2.2 Implementation:                             | 4 |
| 2.2.1 Time Measurement:                         | 5 |
| 3. Result and Discussion:                       | 5 |
| 3.1 Output:                                     | 5 |
| 3.1 Comparison:                                 | 6 |
| 4. The Pseudocode:                              | 6 |
| 4.1 Greedy Activity Selection (Your Algorithm): | 6 |
| 4.2 Brute Force Algorithm:                      | 7 |
| 5. Conclusion:                                  | 7 |

# 1. Introduction:

## 1.1 Overview

In our project we try to create a Task Scheduler system by following the Activity Selection Algorithm. It is a classic problem in algorithm design. Main goal is to select the maximum number of non-overlapping tasks from a given input, where there is a start and end time for every task. In this project we try to use greedy algorithms and brute force methods.

---

## 1.2 Motivation

In real-life scenarios we can solve so many problems by using the Activity Selection Algorithm e.g.: Hospital appointment scheduling, CPU task management, Event planning etc.

It is efficient scheduling is crucial to maximize resource usage and avoid conflicts.

The motivation of the project is to optimize scheduling, understanding how greedy algorithms outperform brute force and to learn practical implementation of algorithm concepts

---

## 1.3 Goals:

The main goals of our project are:

- To implement the Activity Selection Algorithm (Greedy Algorithm).
  - To implement a Brute Force approach so that we can make a comparison
  - To get the maximum number of non-overlapping tasks
  - Comparing efficiency and execution time
  - To verify whether the greedy solution is optimal or not.
-

## 2. Design & Implementation:

### 2.1 Project Details:

Programming Language: C

Input: List of tasks (start time, end time).

Our system is designed to select the maximum number of non-overlapping tasks from a given set of tasks. The system takes user input for multiple tasks and processes them using two different approaches. We use Greedy algorithm to know which selects tasks efficiently and Brute Force method to know which checks all possible combinations and choose the best one.

The purpose of using both methods is to compare their performance and verify whether the greedy solution produces the optimal result without overlapping.

---

### 2.2 Implementation:

The implementation of the project is divided into three steps:

#### **Sorting of Tasks:**

Before applying the greedy algorithm, all tasks are arranged in ascending order based on their end times.

#### **Greedy Scheduling Process:**

Firstly, select the task with the earliest finishing time while the system keeps track of the last selected task. It then checks the next tasks one by one if a task starts after or at the end time of the last selected task, it is selected otherwise, it is skipped.

This process continues until all tasks are checked.

#### **Brute Force Scheduling Process:**

It generates all possible combinations of tasks and for each combination, it checks whether any tasks overlap. From all valid combinations, the one with the maximum number of tasks is selected as the optimal solution.

---

### 2.2.1 Time Measurement:

For the greedy algorithm time complexity is  $O(n^2)$

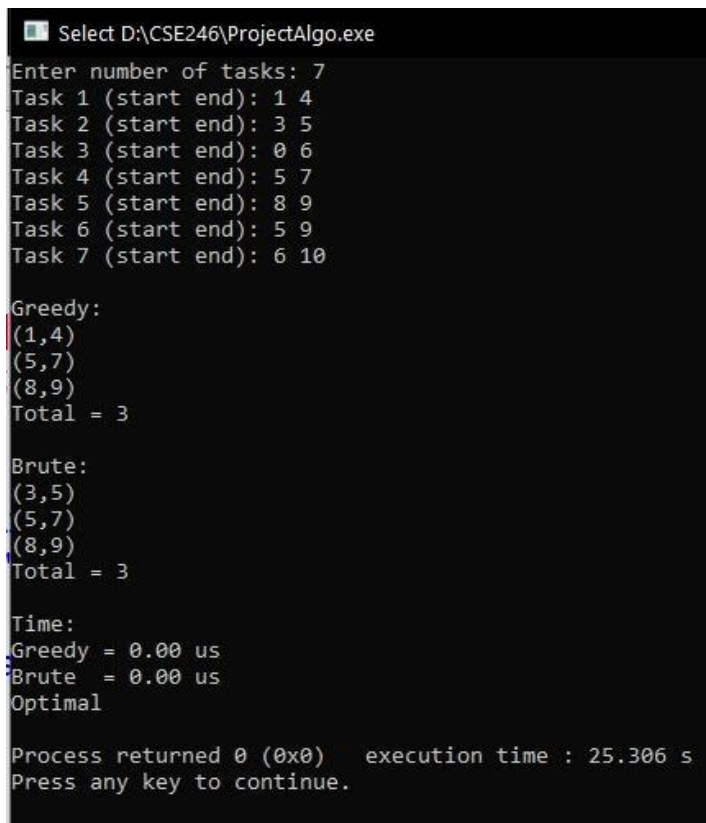
And for the brute force algorithm is  $O(n \times 2^n)$

So, it is  $O(n \times 2^n)$

---

## 3. Result and Discussion:

### 3.1 Output:



```
Select D:\CSE246\ProjectAlgo.exe
Enter number of tasks: 7
Task 1 (start end): 1 4
Task 2 (start end): 3 5
Task 3 (start end): 0 6
Task 4 (start end): 5 7
Task 5 (start end): 8 9
Task 6 (start end): 5 9
Task 7 (start end): 6 10

Greedy:
(1,4)
(5,7)
(8,9)
Total = 3

Brute:
(3,5)
(5,7)
(8,9)
Total = 3

Time:
Greedy = 0.00 us
Brute = 0.00 us
Optimal

Process returned 0 (0x0)   execution time : 25.306 s
Press any key to continue.
```

The program produces possible number of tasks that do not overlap by using the brute force algorithm and choosing the optimal answer by using the greedy algorithm.

---

### 3.1 Comparison:

| Feature          | Greedy Algorithm | Brute Force |
|------------------|------------------|-------------|
| Time Complexity  | $O(n^2)$         | $O(2^n)$    |
| Speed            | Very Fast        | Very Slow   |
| Optimal Solution | Yes              | Yes         |
| Practical Use    | High             | Low         |

---

## 4.The Pseudocode:

### 4.1 Greedy Activity Selection (Your Algorithm):

Greedy-Activity-Selector(tasks)

1. Sort tasks by increasing end time
  2. Select the first task
  3. last\_end = end time of first task
  4. For each remaining task:
    - If task.start  $\geq$  last\_end:
      - Select the task
      - Update last\_end = task.end
  5. Return selected tasks
-

## 4.2 Brute Force Algorithm:

Brute-Force(tasks, index, current\_set)

1. If index == n:

    Check if current\_set has no overlap

    If valid and larger than best\_set:

        Update best\_set

    Return

2. Case 1: Exclude current task

    Call Brute-Force(tasks, index + 1, current\_set)

3. Case 2: Include current task

    Add task[index] to current\_set

    Call Brute-Force(tasks, index + 1, current\_set)

    Remove task[index] from current\_set

---

## 5. Conclusion:

This project successfully demonstrates the implementation of a Task Scheduler using Activity Selection Algorithm. Here, the Greedy approach gives an efficient and optimal result. Using the brute force validates correctness but is computationally expensive. As we did Sorting by end time it make easier to find the optimal greedy selection.

---